

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

ABHISHEK SAVALI(1BM24CS013)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by ABHISHEK SAVALI (1BM24CS013) who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026

. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)	1-21
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	22-25
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling.	26-36
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	37-40
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	41-45
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	46-49
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal.	50-55

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program -1

Question:

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→ SJF (pre-emptive & Non-preemptive)

Code:

=>FCFS:

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
void fcfs(int at[], int bt[], int n) {
    int ct[n], tat[n], wt[n], rt[n], start[n];
    float total_tat = 0, total_wt = 0, total_rt = 0;

    // Sort
    for(int i=0; i<n-1; i++) {
        for(int j=i+1; j<n; j++) {
            if(at[i] > at[j]) {
                int temp;
                temp = at[i]; at[i] = at[j]; at[j] = temp;
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
            }
        }
    }

    // Completion & start times
    start[0] = at[0];
    ct[0] = start[0] + bt[0];
    for(int i=1; i<n; i++) {
        if(at[i] > ct[i-1]) {
```

```

        start[i] = at[i];
    } else {
        start[i] = ct[i-1];
    }
    ct[i] = start[i] + bt[i];
}

// TAT, WT, RT
for(int i=0; i<n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    rt[i] = start[i] - at[i];
    total_tat += tat[i];
    total_wt += wt[i];
    total_rt += rt[i];
}

// Display results
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for(int i=0; i<n; i++) {
    printf("P[%d]\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
}

printf("\nAverage TAT = %.2f", total_tat/n);
printf("\nAverage WT = %.2f", total_wt/n);
printf("\nAverage RT = %.2f\n", total_rt/n);
}

int main() {
    int n;
    printf("Name: AKASH K R\nUSN:1BM24CS027\n\n");
    printf("Enter the number of processes: ");
    scanf("%d", &n);

```

```

int at[n], bt[n];
for(int i=0; i<n; i++) {
    printf("P[%d]\n", i+1);
    printf("AT[%d]: ", i+1);
    scanf("%d", &at[i]);
    printf("BT[%d]: ", i+1);
    scanf("%d", &bt[i]);
}

fcfs(at, bt, n);
return 0;
}

```

Output:

```

C:\Users\Abhis\OneDrive\De  ×  +  v
ABHISHEK SAVALI ---1BM24CS013
Enter number of processes: 4
Arrival time for P1: 0
Burst time for P1: 7
Arrival time for P2: 0
Burst time for P2: 3
Arrival time for P3: 0
Burst time for P3: 4
Arrival time for P4: 0
Burst time for P4: 6

PID      AT      BT      CT      TAT      WT
P1        0       7       7       7       0
P2        0       3      10      10       7
P3        0       4      14      14      10
P4        0       6      20      20      14

Average Turnaround Time = 12.75
Average Waiting Time = 7.75

Gantt Chart:
| P1 | P2 | P3 | P4 |
0       7      10     14     20
Process returned 0 (0x0)   execution time : 42.162 s
Press any key to continue.
|

```

=>SJF(Non-preemptive):

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
void sjf(int at[], int bt[], int n) {
```

```
    int ct[n], tat[n], wt[n], rt[n], start[n], finished[n];
```

```
    float total_tat = 0, total_wt = 0, total_rt = 0;
```

```
    for(int i=0; i<n; i++) {
```

```
        finished[i] = 0;
```

```
    }
```

```
    for(int i=0; i<n-1; i++) {
```

```
        for(int j=i+1; j<n; j++) {
```

```
            if(at[i] > at[j]) {
```

```
                int temp;
```

```
                temp = at[i]; at[i] = at[j]; at[j] = temp;
```

```
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
    int current_time = 0, completed = 0;
```

```
    while(completed < n) {
```

```
        int idx = -1;
```

```
        int min_bt = 99999;
```

```
        for(int i=0; i<n; i++) {
```



```

    if(!finished[i] && at[i] <= current_time) {
        if(bt[i] < min_bt) {
            min_bt = bt[i];
            idx = i;
        }
    }
}

if(idx == -1) {

    current_time++;
} else {

    start[idx] = current_time;

    current_time += bt[idx];
    ct[idx] = current_time;
    tat[idx] = ct[idx] - at[idx];
    wt[idx] = tat[idx] - bt[idx];
    rt[idx] = start[idx] - at[idx];

    total_tat += tat[idx];
    total_wt += wt[idx];
    total_rt += rt[idx];

    finished[idx] = 1;
    completed++;
}
}

```

```

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for(int i=0; i<n; i++) {
    printf("P[%d]\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
}

printf("\nAverage Turnaround Time = %.2f", total_tat/n);
printf("\nAverage Waiting Time = %.2f", total_wt/n);
printf("\nAverage Response Time = %.2f\n", total_rt/n);
}

int main() {
    printf("Name: AKASH K R\nUSN:1BM24CS027\n\n");
    int n;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n];
    for(int i=0; i<n; i++) {
        printf("P[%d]\n", i+1);
        printf("AT[%d]: ", i+1);
        scanf("%d", &at[i]);
        printf("BT[%d]: ", i+1);
        scanf("%d", &bt[i]);
    }

    sjf(at, bt, n);
    return 0;
}

```

Output:

```
"C:\Users\Abhis\OneDrive\De  X + v
ABHISHEK SAVALI ---1BM24CS013
Enter number of processes: 4
Arrival time for P1: 0
Burst time for P1: 7
Arrival time for P2: 8
Burst time for P2: 3
Arrival time for P3: 3
Burst time for P3: 4
Arrival time for P4: 5
Burst time for P4: 6

PID      AT      BT      CT      TAT      WT
P1        0       7       7       7       0
P2        8       3      14       6       3
P3        3       4      11       8       4
P4        5       6      20      15       9

Average Turnaround Time = 9.00
Average Waiting Time = 4.00
Process returned 0 (0x0)    execution time : 24.779 s
Press any key to continue.
```

=>SJF(Pre-emptive)

```
#include <stdio.h>
```

```
void srtf(int n, int at[], int bt[])  
{  
    int rt[n], ct[n], tat[n], wt[n];  
    int time = 0, done = 0;  
    float total_wt = 0, total_tat = 0;
```

```
    for(int i = 0; i < n; i++)  
        rt[i] = bt[i];
```

```
    while(done < n)  
    {  
        int pos = -1;  
        int min = 9999;
```

```
        for(int i = 0; i < n; i++)  
        {  
            if(at[i] <= time && rt[i] > 0)  
            {  
                if(rt[i] < min)  
                {  
                    min = rt[i];  
                    pos = i;  
                }  
            }  
        }  
    }
```

```
    if(pos == -1)
```

```

    {
        time++;
    }
else
{
    rt[pos]--;
    time++;

    if(rt[pos] == 0)
    {
        done++;

        ct[pos] = time;
        tat[pos] = ct[pos] - at[pos];
        wt[pos] = tat[pos] - bt[pos];

        total_wt += wt[pos];
        total_tat += tat[pos];
    }
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

for(int i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

```

```

    printf("\nAverage Waiting Time = %.2f", total_wt/n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat/n);
}

int main()
{
    printf("Name: AKASH K R\nUSN: 1BM24CS027\n");

    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n];

    for(int i = 0; i < n; i++)
    {
        printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    srtf(n, at, bt);

    return 0;
}

```

Output:

```
"C:\Users\Abhis\OneDrive\De  ×  +  v
ABHISHEK SAVALI ---1BM24CS013
Enter number of processes: 4
Arrival time for P1: 0
Burst time for P1: 7
Arrival time for P2: 8
Burst time for P2: 3
Arrival time for P3: 3
Burst time for P3: 2
Arrival time for P4: 5
Burst time for P4: 6

PID      AT      BT      CT      TAT      WT
P1        0       7       9       9        2
P2        8       3      12       4        1
P3        3       2       5       2        0
P4        5       6      18      13        7

Average Turnaround Time = 7.00
Average Waiting Time = 2.50
Process returned 0 (0x0)   execution time : 23.217 s
Press any key to continue.
```

=> Priority (pre-emptive)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
void pri_pre(int at[],int bt[],int p[],int rt[],int n)
```

```
{
```

```
    int time=0,comp=0;
```

```
    int ct[n],tat[n],wt[n];
```

```
    float avg_tat=0,avg_wt=0;
```

```
    while(comp<n)
```

```
    {
```

```
        int pri=999,idx=-1;
```

```
        for(int i=0;i<n;i++)
```

```
        {
```

```
            if(at[i]<=time && rt[i]>0 && p[i]<pri)
```

```
            {
```

```
                pri=p[i];
```

```
                idx=i;
```

```
            }
```

```
        }
```

```
        if(idx!=-1)
```

```
        {
```

```
            time++;
```

```
        }
```

```
    else{
```

```
        rt[idx]--;
```

```
        time++;
```

```
        if(rt[idx]==0){
```

```
            ct[idx]=time;
```

```
            tat[idx]=ct[idx]-at[idx];
```

```
            wt[idx]=tat[idx]-bt[idx];
```

```
            comp++;
```



```

        }
    }
}
printf("\nPID\tAT\tBT\tP\tCT\tTAT\tWT\n");
for(int i=0;i<n;i++)
{
    printf("P[%d]\t%d\t%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],p[i],ct[i],tat[i],wt[i]);
    avg_tat+=tat[i];
    avg_wt+=wt[i];
}
printf("Avg TAT: %.2f\n",avg_tat/n);
printf("Avg WT: %.2f",avg_wt/n);
}

int main()
{
    printf("USN: 1BM24CS027\n");
    int n;
    printf("Enter the number of processes: ");
    scanf("%d",&n);
    int at[n],bt[n],p[n],rt[n];
    printf("Enter the values of AT,BT,Priority\n");
    for(int i=0;i<n;i++)
    {
        printf("p[%d]:",i+1);
        scanf("%d%d%d",&at[i],&bt[i],&p[i]);
        rt[i]=bt[i];
    }
    pri_pre(at,bt,p,rt,n);
    return 0;
}

```

Output :

```
"C:\Users\Abhis\OneDrive\De  X + v
ABHISHEK SAVALI ---1BM24CS013
Enter number of processes: 5

Arrival time for P1: 0
Burst time for P1: 3
Priority for P1: 5

Arrival time for P2: 2
Burst time for P2: 2
Priority for P2: 3

Arrival time for P3: 3
Burst time for P3: 5
Priority for P3: 2

Arrival time for P4: 4
Burst time for P4: 4
Priority for P4: 4

Arrival time for P5: 6
Burst time for P5: 1
Priority for P5: 1

PID    AT    BT    PR    CT    TAT    WT
P1      0     3     5    15    15    12
P2      2     2     3    10     8     6
P3      3     5     2     9     6     1
P4      4     4     4    14    10     6
P5      6     1     1     7     1     0

Average Turnaround Time = 8.00
Average Waiting Time = 5.00

Gantt Chart:
| P1 | P2 | P3 | P5 | P3 | P2 | P4 | P1 |
0    2    3    6    7    9    10   14   15
Process returned 0 (0x0)  execution time : 41.103 s
Press any key to continue.
|
```

=> Priority (Non-pre-emptive)

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
void pri_npre(int at[],int bt[],int p[],int n)
```

```
{
```

```
    int ct[n],tat[n],wt[n],completed[n];
```

```
    int time=0,count=0;
```

```
    float avg_tat=0,avg_wt=0;
```

```
    for(int i=0;i<n;i++)
```

```
    {
```

```
        completed[i]=0;
```

```
    }
```

```
    while(count<n)
```

```
    {
```

```
        int pri=999,idx=-1;
```

```
        for(int i=0;i<n;i++)
```

```
        {
```

```
            if(at[i]<=time && completed[i]==0 && p[i]<pri)
```

```
            {
```

```
                pri=p[i];
```

```
                idx=i;
```

```
            }
```

```
        }
```

```
        if(idx== -1)
```

```
        {
```

```
            time++;
```

```
        }
```

```
        else{
```

```
            ct[idx]=time+bt[idx];
```

```
            tat[idx]=ct[idx]-at[idx];
```

```
            wt[idx]=tat[idx]-bt[idx];
```

```

        time=ct[idx];
        completed[idx]=1;
        count++;
    }
}
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for(int i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],ct[i],tat[i],wt[i]);
    avg_tat+=tat[i];
    avg_wt+=wt[i];
}
printf("Avg TAT: %.2f:",avg_tat/n);
printf("Avg WT: %.2f:",avg_wt/n);
}

int main()
{
    int n;
    printf("Enter the number of processes: ");
    scanf("%d",&n);
    int at[n],bt[n],p[n];
    printf("Enter the AT,BT and Priority\n");
    for(int i=0;i<n;i++)
    {
        printf("p[%d] : ",i+1);
        scanf("%d%d%d",&at[i],&bt[i],&p[i]);
    }
    pri_npre(at,bt,p,n);
    return 0;
}

```

Output :

```
"C:\Users\Abhis\OneDrive\De  X + v
ABHISHEK SAVALI ---1BM24CS013
Enter number of processes: 5

Arrival time for P1: 0
Burst time for P1: 3
Priority for P1: 5

Arrival time for P2: 2
Burst time for P2: 2
Priority for P2: 3

Arrival time for P3: 3
Burst time for P3: 5
Priority for P3: 2

Arrival time for P4: 4
Burst time for P4: 4
Priority for P4: 4

Arrival time for P5: 6
Burst time for P5: 1
Priority for P5: 1

PID      AT      BT      PR      CT      TAT      WT
P1        0        3        5        3        3        0
P2        2        2        3       11        9        7
P3        3        5        2        8        5        0
P4        4        4        4       15       11        7
P5        6        1        1        9        3        2

Average Turnaround Time = 6.20
Average Waiting Time = 3.20

Gantt Chart:
| P1 | P3 | P5 | P2 | P4 |
0    3    8    9   11   15
Process returned 0 (0x0)   execution time : 47.533 s
Press any key to continue.
|
```

=>Round Robin

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void round_robin(int at[], int bt[], int n, int quantum) {
```

```
    int rt_remaining[n], ct[n], tat[n], wt[n], rt[n];
```

```
    int first_visit[n];
```

```
    int time = 0, count = 0;
```

```
    float avg_tat = 0, avg_wt = 0;
```

```
    int queue[1000], head = 0, tail = 0;
```

```
    int in_queue[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        rt_remaining[i] = bt[i];
```

```
        in_queue[i] = 0;
```

```
        first_visit[i] = 1;
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (at[i] <= time) {
```

```
            queue[tail++] = i;
```

```
            in_queue[i] = 1;
```

```
        }
```

```
    }
```

```
    while (count < n) {
```

```
        if (head == tail) {
```

```
            time++;
```

```
            for (int i = 0; i < n; i++) {
```

```
                if (at[i] <= time && rt_remaining[i] > 0 && in_queue[i] == 0) {
```

```
                    queue[tail++] = i;
```

```
                    in_queue[i] = 1;
```

```

    }
}
continue;
}

int idx = queue[head++];

if (first_visit[idx]) {
    rt[idx] = time - at[idx];
    first_visit[idx] = 0;
    avg_rt += rt[idx];
}
if (rt_remaining[idx] > quantum) {
    time += quantum;
    rt_remaining[idx] -= quantum;
} else {
    time += rt_remaining[idx];
    rt_remaining[idx] = 0;
    ct[idx] = time;
    tat[idx] = ct[idx] - at[idx];
    wt[idx] = tat[idx] - bt[idx];
    count++;
}

for (int i = 0; i < n; i++) {
    if (at[i] <= time && rt_remaining[i] > 0 && in_queue[i] == 0) {
        queue[tail++] = i;
        in_queue[i] = 1;
    }
}

if (rt_remaining[idx] > 0) {
    queue[tail++] = idx;
}

```

```

    } else {
        in_queue[idx] = 0;
    }
}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
    avg_tat += tat[i];
    avg_wt += wt[i];
}
printf("\nAvg TAT: %.2f", avg_tat / n);
printf("\nAvg WT: %.2f", avg_wt / n);
}

int main() {
    int n, quantum;
    printf("USN: 1BM24CS027\n");
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter Time Quantum: ");
    scanf("%d", &quantum);

    int at[n], bt[n];
    printf("Enter the AT and BT\n");
    for (int i = 0; i < n; i++) {
        printf("p[%d] : ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    round_robin(at, bt, n, quantum);
    return 0;
}

```


Output :

```
"C:\Users\Abhis\OneDrive\De  X + v
ABHISHEK SAVALI ---1BM24CS013
Enter number of processes: 5

Arrival time for P1: 0
Burst time for P1: 5

Arrival time for P2: 1
Burst time for P2: 3

Arrival time for P3: 2
Burst time for P3: 1

Arrival time for P4: 3
Burst time for P4: 2

Arrival time for P5: 4
Burst time for P5: 3

Enter Time Quantum: 2

PID    AT    BT    CT    TAT    WT
P3      2     1     5     3     2
P4      3     2     7     4     2
P2      1     3    12    11     8
P5      4     3    13     9     6
P1      0     5    14    14     9

Average Turnaround Time = 8.20
Average Waiting Time = 5.40

Gantt Chart:
| P1 | P2 | P3 | P4 | P5 | P1 | P2 | P5 | P1 |
0      2      4      5      7      9     11     12     13     14
Process returned 0 (0x0)  execution time : 30.047 s
Press any key to continue.
|
```

Program – 2

Question :

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE :

```
#include<stdio.h>
#include<stdlib.h>

void mlq_scheduling(int at[], int bt[], int q_no[], int n, int tq) {
    int rt[n], ct[n], tat[n], wt[n], completed[n];
    int time = 0, count = 0, last_q1 = -1;
    float avg_tat = 0, avg_wt = 0;

    for(int i = 0; i < n; i++) {
        rt[i] = bt[i];
        completed[i] = 0;
    }

    while(count < n) {
        int idx = -1;

        //Queue 1-Round Robin
        for(int i = 1; i <= n; i++) {
            int j = (last_q1 + i) % n;
            if(q_no[j] == 1 && at[j] <= time && completed[j] == 0) {
                idx = j;
                break;
            }
        }

        if(idx != -1) {
```

```

int slice = (rt[idx] > tq) ? tq : rt[idx];
time += slice;
rt[idx] -= slice;
last_q1 = idx;

if(rt[idx] == 0) {
    ct[idx] = time;
    completed[idx] = 1;
    count++;
}
}
else {
    //Queue 2-FCFS
    for(int i = 0; i < n; i++) {
        if(q_no[i] == 2 && at[i] <= time && completed[i] == 0) {
            idx = i;
            break;
        }
    }

    if(idx != -1) {
        rt[idx]--;
        time++;
        if(rt[idx] == 0) {
            ct[idx] = time;
            completed[idx] = 1;
            count++;
        }
    } else {
        time++;
    }
}
}

```

```

printf("\nPID\tQ_No\tAT\tBT\tCT\tTAT\tWT\n");
for(int i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, q_no[i], at[i], bt[i], ct[i], tat[i], wt[i]);
    avg_tat += tat[i];
    avg_wt += wt[i];
}
printf("\nAvg TAT: %.2f", avg_tat/n);
printf("\nAvg WT: %.2f\n", avg_wt/n);
}

int main() {
    printf("USN:1BM24CS027\n");
    int n, tq = 2;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], q_no[n];
    printf("Enter the Queue_No, AT and BT\n");
    for(int i = 0; i < n; i++) {
        printf("p[%d] : ", i+1);
        scanf("%d %d %d", &q_no[i], &at[i], &bt[i]);
    }

    mlq_scheduling(at, bt, q_no, n, tq);

    return 0;
}

```

Output :

```
"C:\Users\Abhis\OneDrive\De  × + v
ABHISHEK SAVALI ---1BM24CS013
Enter number of processes: 5

Arrival Time for P1: 0
Burst Time for P1: 4
Queue Priority for P1 (1-System / 2-User): 1

Arrival Time for P2: 1
Burst Time for P2: 3
Queue Priority for P2 (1-System / 2-User): 2

Arrival Time for P3: 2
Burst Time for P3: 2
Queue Priority for P3 (1-System / 2-User): 1

Arrival Time for P4: 3
Burst Time for P4: 1
Queue Priority for P4 (1-System / 2-User): 2

Arrival Time for P5: 4
Burst Time for P5: 3
Queue Priority for P5 (1-System / 2-User): 1

PID      AT      BT      Queue  CT      TAT      WT
P1        0        4        1        4        4        0
P2        1        3        2       12       11        8
P3        2        2        1        6        4        2
P4        3        1        2       13       10        9
P5        4        3        1        9        5        2

Average Turnaround Time = 6.80
Average Waiting Time = 4.20

Gantt Chart:
| P1 | P3 | P5 | P2 | P4 |
0    4    6    9    12    13
Process returned 0 (0x0)  execution time : 51.733 s
Press any key to continue.
|
```

Program – 3

Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a) Rate- Monotonic
- b) Earliest-deadline First
- c)Proportional scheduling

CODE :

=> Rate- Monotonic

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int gcd(int a, int b) {
```

```
    if (b == 0) return a;
```

```
    return gcd(b, a % b);
```

```
}
```

```
int lcm(int a, int b) {
```

```
    return (a * b) / gcd(a, b);
```

```
}
```

```
void rms(int cap[], int period[], int n) {
```

```
    int hyper = period[0];
```

```
    for (int i = 1; i < n; i++) {
```

```
        hyper = lcm(hyper, period[i]);
```

```
    }
```

```
    printf("LCM=%d\n\n", hyper);
```

```
    printf("---Rate Monotonic Scheduling---\n");
```

```
    printf("PID\tcapacity\tPeriod\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d\t%d\t%d\n", i+1, cap[i], period[i]);
```

```
    }
```

```

printf("\nScheduling occurs for %d ms\n\n", hyper);

int remaining[n];
for (int i = 0; i < n; i++) remaining[i] = 0;

int last = -2;

for (int t = 0; t < hyper; t++) {
    for (int i = 0; i < n; i++) {
        if (t % period[i] == 0) {
            remaining[i] = cap[i];
        }
    }
}

int chosen = -1;
for (int i = 0; i < n; i++) {
    if (remaining[i] > 0) {
        if (chosen == -1 || period[i] < period[chosen]) {
            chosen = i;
        }
    }
}

if (chosen != -1) {
    if (chosen != last) {
        printf("%dms onwards: Process %d running\n", t, chosen+1);
        last = chosen;
    }
    remaining[chosen]--;
} else {
    if (last != -1) {
        printf("%dms onwards: CPU is idle\n", t);
        last = -1;
    }
}

```

```

    }
}
}
}
int main() {
    printf("USN: 1BM24CS027\n");
    int n;
    double sum = 0;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

    int cap[n], period[n];
    printf("Enter capacity,period:\n");
    for (int i = 0; i < n; i++) {
        printf("p[%d] : ", i+1);
        scanf("%d %d", &cap[i], &period[i]);
        sum += (double)cap[i] / period[i];
    }

    if (sum <= 1) {
        rms(cap, period, n);
    } else {
        printf("Not possible\n");
    }
    return 0;
}

```


Output :

```
"C:\Users\Abhis\OneDrive\De  ×  +  v
ABHISHEK SAVALI ---1BM24CS013
Enter number of tasks: 3

Task 1
Execution time (C1): 3
Period (P1): 20

Task 2
Execution time (C2): 2
Period (P2): 5

Task 3
Execution time (C3): 2
Period (P3): 10

--- RMS Scheduling ---
Time 0 : Task 2
Time 1 : Task 2
Time 2 : Task 3
Time 3 : Task 3
Time 4 : Task 1
Time 5 : Task 2
Time 6 : Task 2
Time 7 : Task 1
Time 8 : Task 1
Time 9 : Idle
Time 10 : Task 2
Time 11 : Task 2
Time 12 : Task 3
Time 13 : Task 3
Time 14 : Idle
Time 15 : Task 2
Time 16 : Task 2
Time 17 : Idle
Time 18 : Idle
Time 19 : Idle

Process returned 0 (0x0)   execution time : 15.588 s
Press any key to continue.
|
```

=> Earliest-deadline First

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <limits.h>
```

```
int gcd(int a, int b) {  
    if (b == 0) return a;  
    return gcd(b, a % b);  
}
```

```
int lcm(int a, int b) {  
    return (a * b) / gcd(a, b);  
}
```

```
void edf(int cap[], int dead[], int period[], int n)  
{  
    int hyper = period[0];  
    for (int i = 1; i < n; i++) {  
        hyper = lcm(hyper, period[i]);  
    }  
  
    printf("\nScheduling occurs for %d ms\n\n", hyper);
```

```
    int remaining[n];
```

```
    int abs_dead[n];
```

```
    for (int i = 0; i < n; i++) {  
        remaining[i] = 0;  
        abs_dead[i] = dead[i];  
    }
```

```
    int last = -2;
```

```

for (int t = 0; t < hyper; t++)
{
    for (int i = 0; i < n; i++) {
        if (t % period[i] == 0) {
            remaining[i] += cap[i];
            abs_dead[i] = t + dead[i];
        }
    }

    int chosen = -1;
    int min_dead = INT_MAX;

    for (int i = 0; i < n; i++) {
        if (remaining[i] > 0 && abs_dead[i] < min_dead) {
            min_dead = abs_dead[i];
            chosen = i;
        }
    }

    if (chosen != -1) {
        if (last != chosen) {
            printf("%dms onwards: Task %d is running\n", t, chosen+1);
            last = chosen;
        }
        remaining[chosen]--;
    } else {
        if (last != -1) {
            printf("%dms onwards: CPU is idle\n", t);
            last = -1;
        }
    }
}

```

```

int main()
{
    printf("USN:1BM24CS027\n");
    int n;
    double sum = 0;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    int cap[n], dead[n], period[n];
    printf("Enter the capacity, deadline, period\n");
    for (int i = 0; i < n; i++)
    {
        printf("P[%d]: ", i+1);
        scanf("%d %d %d", &cap[i], &dead[i], &period[i]);

        sum += (double)cap[i] / period[i];
    }

    if (sum <= 1)
    {
        edf(cap, dead, period, n);
    }
    else
    {
        printf("Not possible\n");
    }

    return 0;
}

```

Output :

```
"C:\Users\Abhis\OneDrive\De  X + v
ABHISHEK SAVALI ---1BM24CS013
Enter number of tasks: 3

Task 1
Execution Time (C1): 3
Deadline (D1): 7
Time Period (P1): 20

Task 2
Execution Time (C2): 2
Deadline (D2): 4
Time Period (P2): 5

Task 3
Execution Time (C3): 2
Deadline (D3): 8
Time Period (P3): 10

Enter Total Scheduling Time: 20

--- EDF Scheduling (Periodic Tasks) ---
Time 0 : Task 2
Time 1 : Task 2
Time 2 : Task 1
Time 3 : Task 1
Time 4 : Task 1
Time 5 : Task 3
Time 6 : Task 3
Time 7 : Task 2
Time 8 : Task 2
Time 9 : Idle
Time 10 : Task 2
Time 11 : Task 2
Time 12 : Task 3
Time 13 : Task 3
Time 14 : Idle
Time 15 : Task 2
Time 16 : Task 2
Time 17 : Idle
Time 18 : Idle
Time 19 : Idle
```

=> Proportional scheduling

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
typedef struct {
```

```
    char name[5];
```

```
    int tickets;
```

```
} Process;
```

```
int main() {
```

```
    printf("USN: 1BM24CS027\n");
```

```
    int n, total_tickets = 0;
```

```
    float total_T = 0.0;
```

```
    printf("Enter the number of Processes: ");
```

```
    scanf("%d", &n);
```

```
    Process p[n];
```

```
    srand(time(NULL));
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("\nProcess %d:\n", i + 1);
```

```
        sprintf(p[i].name, "P%d", i + 1);
```

```
        printf("Tickets: ");
```

```
        scanf("%d", &p[i].tickets);
```

```
        total_tickets += p[i].tickets;
```

```
        total_T += p[i].tickets;
```

```
    }
```

```
    printf("\n--- Proportional Share Scheduling ---\n");
```

```
    printf("Enter the Time Period for scheduling: ");
```

```
    int m;
```

```
    scanf("%d", &m);
```

```

for (int i = 0; i < m; i++) {
    int winning_ticket = rand() % total_tickets + 1;
    int accumulated_tickets = 0;
    int winner_index;

    for (int j = 0; j < n; j++) {
        accumulated_tickets += p[j].tickets;
        if (winning_ticket <= accumulated_tickets) {
            winner_index = j;
            break;
        }
    }
    printf("Tickets picked: %d, Winner: %s\n",
        winning_ticket, p[winner_index].name);
}

for (int i = 0; i < n; i++) {
    printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n",
        p[i].name, ((p[i].tickets / total_T) * 100));
}

return 0;
}

```

Output :

```
USN: 1BM24CS013
Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 16, Winner: P2
Tickets picked: 37, Winner: P3
Tickets picked: 27, Winner: P2
Tickets picked: 60, Winner: P3
Tickets picked: 20, Winner: P2

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.
```

ChatGPT

Program – 4

Question :

Write a C program to simulate:

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

Code :

=> producer-consumer

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty;
sem_t full;
pthread_mutex_t mutex;
void* producer(void* arg) {
    int item;
    for (int i = 0; i < 10; i++) {
        item = i;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Produced: %d at buffer[%d]\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        sleep(1);
    }
    return NULL;
}
void* consumer(void* arg) {
    int item;
    for (int i = 0; i < 10; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
```

```

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        sleep(2);
    }
    return NULL;
}

int main() {
    printf("USN: 1BM24CS027\n");

    pthread_t prod, cons;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

Output :

```

C:\Users\Abhis\OneDrive\De
ABHISHEK SAVALI ---1BM24CS013
Produced : 0 at buffer[0]
Consumed : 0 from buffer[0]
Produced : 0 at buffer[0]
Consumed : 0 from buffer[0]
Produced : 0 at buffer[0]

```

=> Dining Philosophers

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#define N 5
```

```
pthread_mutex_t forks[N];
```

```
pthread_t philosophers[N];
```

```
void* philosopher(void* num) {
```

```
    int id = *(int*)num;
```

```
    int left = id;
```

```
    int right = (id + 1) % N;
```

```
    while (1) {
```

```
        printf("Philosopher %d is thinking.\n", id);
```

```
        sleep(1);
```

```
        if (id % 2 == 0) {
```

```
            pthread_mutex_lock(&forks[left]);
```

```
            printf("Philosopher %d picked up left fork %d.\n", id, left);
```

```
            pthread_mutex_lock(&forks[right]);
```

```
            printf("Philosopher %d picked up right fork %d.\n", id, right);
```

```
        } else {
```

```
            pthread_mutex_lock(&forks[right]);
```

```
            printf("Philosopher %d picked up right fork %d.\n", id, right);
```

```
            pthread_mutex_lock(&forks[left]);
```

```
            printf("Philosopher %d picked up left fork %d.\n", id, left);
```

```
        }
```

```
        printf("Philosopher %d is eating.\n", id);
```

```
        sleep(2);
```

```
        pthread_mutex_unlock(&forks[left]);
```

```
        pthread_mutex_unlock(&forks[right]);
```

```
        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
```

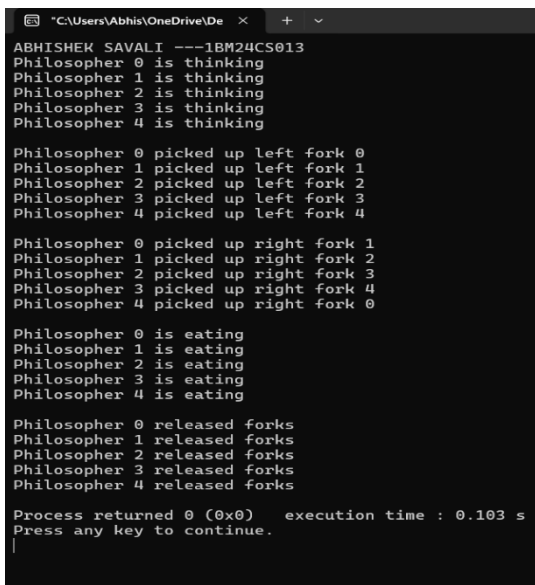
```
    }
```

```

    return NULL;
}
int main() {
    int i;
    int ids[N];
    for (i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }
    for (i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }
    for (i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }
    for (i = 0; i < N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }
    return 0;
}

```

Output :



```

ABHISHEK SAVALI ---1BM24CS013
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking

Philosopher 0 picked up left fork 0
Philosopher 1 picked up left fork 1
Philosopher 2 picked up left fork 2
Philosopher 3 picked up left fork 3
Philosopher 4 picked up left fork 4

Philosopher 0 picked up right fork 1
Philosopher 1 picked up right fork 2
Philosopher 2 picked up right fork 3
Philosopher 3 picked up right fork 4
Philosopher 4 picked up right fork 0

Philosopher 0 is eating
Philosopher 1 is eating
Philosopher 2 is eating
Philosopher 3 is eating
Philosopher 4 is eating

Philosopher 0 released forks
Philosopher 1 released forks
Philosopher 2 released forks
Philosopher 3 released forks
Philosopher 4 released forks

Process returned 0 (0x0)   execution time : 0.103 s
Press any key to continue.

```

Program – 5

Question:

Write a C program to simulate: (Any one)

- a) Bankers' algorithm for the purpose of deadlock avoidance.
- b) Deadlock Detection

CODE :

=> Banker's

```
#include <stdio.h>
```

```
void bankersSafety(int n, int m, int alloc[n][m], int max[n][m], int avail[m]) {  
    int need[n][m], finish[n], safeSeq[n];  
    int i, j, k;  
  
    for (i = 0; i < n; i++) {  
        for (j = 0; j < m; j++)  
            need[i][j] = max[i][j] - alloc[i][j];  
        finish[i] = 0;  
    }  
  
    int count = 0;  
    while (count < n) {  
        int found = 0;  
        for (i = 0; i < n; i++) {  
            if (finish[i] == 0) {  
                int flag = 1;  
                for (j = 0; j < m; j++) {  
                    if (need[i][j] > avail[j]) {  
                        flag = 0;  
                        break;  
                    }  
                }  
                if (flag) {  
                    safeSeq[count] = i;  
                    finish[i] = 1;  
                    for (j = 0; j < m; j++)  
                        avail[j] += alloc[i][j];  
                    count++;  
                }  
            }  
        }  
    }  
}
```

```

        for (k = 0; k < m; k++)
            avail[k] += alloc[i][k];
        safeSeq[count++] = i;
        finish[i] = 1;
        found = 1;
    }
}
}
if (!found) {
    printf("System is not in a safe state.\n");
    return;
}
}

printf("System is in a safe state.\nSafe sequence is: ");
for (i = 0; i < n; i++)
    printf("P%d -> ", safeSeq[i]);
printf("\n");
}

int main() {
    printf("USN : 1BM24CS027\n\n");
    int n, m, i, j;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], avail[m];

    printf("Enter Allocation Matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)

```

```

scanf("%d", &alloc[i][j]);

printf("Enter Maximum Demand Matrix:\n");
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        scanf("%d", &max[i][j]);

printf("Enter Available Resources:\n");
for (j = 0; j < m; j++)
    scanf("%d", &avail[j]);

bankersSafety(n, m, alloc, max, avail);

return 0;
}

```

Output :

```

ABHISHEK SAVALI ---1BM24CS013
Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

System is in SAFE STATE
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
Process returned 0 (0x0)    execution time : 135.947 s
Press any key to continue.
|

```

=>Deadlock Detection

```
#include <stdio.h>
```

```
void deadlockDetection(int n, int m, int alloc[n][m], int request[n][m], int avail[m]) {
    int finish[n], safeSeq[n];
    int i, j, k, count = 0;

    for (i = 0; i < n; i++)
        finish[i] = 0;

    while (count < n) {
        int found = 0;
        for (i = 0; i < n; i++) {
            if (finish[i] == 0) {
                int flag = 1;
                for (j = 0; j < m; j++) {
                    if (request[i][j] > avail[j]) {
                        flag = 0;
                        break;
                    }
                }
                if (flag) {
                    for (k = 0; k < m; k++)
                        avail[k] += alloc[i][k];
                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if (!found) {
            printf("System is not in a safe state (Deadlock detected).\n");
            return;
        }
    }

    printf("System is in safe state.\nSafe Sequence is: ");
    for (i = 0; i < n; i++)
        printf("P%d ", safeSeq[i]);
    printf("\n");
}
```

```
int main() {
    printf("USN: 1BM24CS027\n");
    int n, m, i, j;
    printf("Enter the number of processes: ");
```



```

scanf("%d", &n);
printf("Enter the number of resources: ");
scanf("%d", &m);

int alloc[n][m], request[n][m], avail[m];

printf("Enter the allocation matrix:\n");
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        scanf("%d", &alloc[i][j]);

printf("Enter the request matrix:\n");
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        scanf("%d", &request[i][j]);

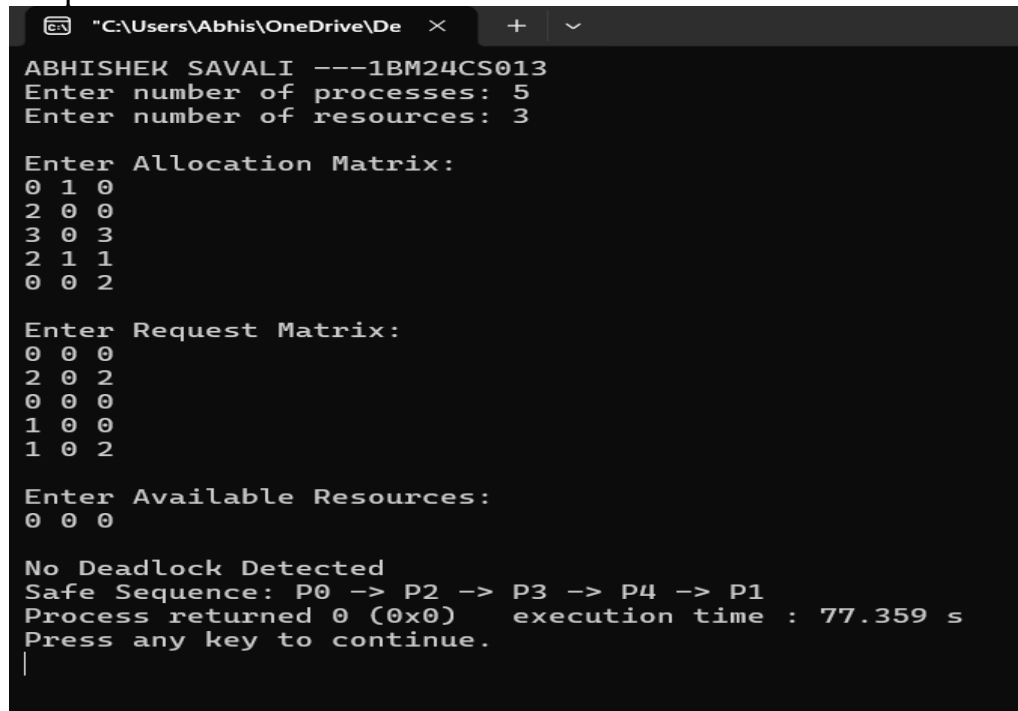
printf("Enter the available resources:\n");
for (j = 0; j < m; j++)
    scanf("%d", &avail[j]);

deadlockDetection(n, m, alloc, request, avail);

return 0;
}

```

Output :



```

ABHISHEK SAVALI ---1BM24CS013
Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Enter Request Matrix:
0 0 0
2 0 2
0 0 0
1 0 0
1 0 2

Enter Available Resources:
0 0 0

No Deadlock Detected
Safe Sequence: P0 -> P2 -> P3 -> P4 -> P1
Process returned 0 (0x0)    execution time : 77.359 s
Press any key to continue.
|

```

Program – 6

Question :

Write a C program to simulate the following contiguous memory allocation techniques. (Any one)

- a) Worst-fit
- b) Best-fit
- c) First-fit

CODE :

=> Worst-fit, Best-fit and First-fit

```
#include <stdio.h>
```

```
void firstFit(int blocks[], int m, int processes[], int n) {
    int allocation[n];
    int used[m];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < m; i++)
        used[i] = 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {

            // allocate only if block is unused
            if (!used[j] && blocks[j] >= processes[i]) {
                allocation[i] = j;
                used[j] = 1;
                break;
            }
        }
    }

    printf("\n--- First Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t", i + 1, processes[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}
```

```

}

void bestFit(int blocks[], int m, int processes[], int n) {
    int allocation[n];
    int used[m];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < m; i++)
        used[i] = 0;

    for (int i = 0; i < n; i++) {
        int best = -1;

        for (int j = 0; j < m; j++) {

            // find smallest suitable unused block
            if (!used[j] && blocks[j] >= processes[i]) {
                if (best == -1 || blocks[j] < blocks[best]) {
                    best = j;
                }
            }
        }

        if (best != -1) {
            allocation[i] = best;
            used[best] = 1;
        }
    }

    printf("\n--- Best Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t", i + 1, processes[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int blocks[], int m, int processes[], int n) {
    int allocation[n];
    int used[m];

```

```

for (int i = 0; i < n; i++)
    allocation[i] = -1;

for (int i = 0; i < m; i++)
    used[i] = 0;

for (int i = 0; i < n; i++) {
    int worst = -1;

    for (int j = 0; j < m; j++) {

        if (!used[j] && blocks[j] >= processes[i]) {
            if (worst == -1 || blocks[j] > blocks[worst]) {
                worst = j;
            }
        }
    }

    if (worst != -1) {
        allocation[i] = worst;
        used[worst] = 1;
    }
}

printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for (int i = 0; i < n; i++) {
    printf("%d\t%d\t", i + 1, processes[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

}

int main() {
    printf("USN: 1BM24CS027\n");
    int m, n;
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int blocks[m];

    printf("Enter sizes of %d memory blocks:\n", m);

```

```

for (int i = 0; i < m; i++) {
    scanf("%d", &blocks[i]);
}

printf("Enter number of processes: ");
scanf("%d", &n);

int processes[n];

printf("Enter sizes of %d processes:\n", n);
for (int i = 0; i < n; i++) {
    scanf("%d", &processes[i]);
}

firstFit(blocks, m, processes, n);
bestFit(blocks, m, processes, n);
worstFit(blocks, m, processes, n);

return 0;
}

```

Output :

```

C:\Users\Abhis\OneDrive\De
ABHISHEK SAVALI ---1BM24CS013
Enter number of memory blocks: 5
Enter size of each memory block:
400
700
200
300
600
Enter number of processes: 4
Enter size of each process:
212
512
312
526

===== FIRST FIT =====
Process Size      Block No
P1      212        1
P2      512        2
P3      312        5
P4      526        Not Allocated

===== BEST FIT =====
Process Size      Block No
P1      212        4
P2      512        5
P3      312        1
P4      526        2

===== WORST FIT =====
Process Size      Block No
P1      212        2
P2      512        5
P3      312        2
P4      526        Not Allocated

Process returned 0 (0x0)   execution time : 33.493 s
Press any key to continue.
|

```

Program -7

Question:

Write a C program to simulate page replacement algorithms.

- a) FIFO
- b) LRU
- c) Optimal

CODE:

=> FIFO, LRU and Optimal

```
#include <stdio.h>
```

```
int pages[50], frames[10];
```

```
/* ----- FIFO ----- */
```

```
void FIFO(int n, int f)
```

```
{
```

```
    int i, j, found, faults = 0;
```

```
    int next = 0;
```

```
    for(i = 0; i < f; i++)
```

```
        frames[i] = -1;
```

```
    printf("\nFIFO Page Replacement\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        found = 0;
```

```
        for(j = 0; j < f; j++)
```

```
        {
```

```
            if(frames[j] == pages[i])
```

```
            {
```

```
                found = 1;
```

```
                break;
```

```
            }
```

```
        }
```

```
    if(found == 0)
```

```
    {
```

```
        frames[next] = pages[i];
```

```
        next = (next + 1) % f;
```

```
        faults++;
```

```
    }
```

```
    printf("Page %d -> ", pages[i]);
```

```

        for(j = 0; j < f; j++)
        {
            if(frames[j] != -1)
                printf("%d ", frames[j]);
        }

        printf("\n");
    }

    printf("Total Page Faults = %d\n", faults);
}

/* ----- LRU ----- */
void LRU(int n, int f)
{
    int i, j, found, faults = 0;
    int time[10], counter = 0;
    int least, pos;

    for(i = 0; i < f; i++)
    {
        frames[i] = -1;
        time[i] = 0;
    }

    printf("\nLRU Page Replacement\n");

    for(i = 0; i < n; i++)
    {
        found = 0;

        for(j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }

        if(found == 0)
        {
            least = time[0];
            pos = 0;

```

```

        for(j = 1; j < f; j++)
        {
            if(time[j] < least)
            {
                least = time[j];
                pos = j;
            }
        }

        counter++;
        frames[pos] = pages[i];
        time[pos] = counter;
        faults++;
    }

    printf("Page %d -> ", pages[i]);

    for(j = 0; j < f; j++)
    {
        if(frames[j] != -1)
            printf("%d ", frames[j]);
    }

    printf("\n");
}

printf("Total Page Faults = %d\n", faults);
}

/* ----- OPTIMAL ----- */
void OPTIMAL(int n, int f)
{
    int i, j, k, found, faults = 0;
    int pos, farthest, index;

    for(i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nOptimal Page Replacement\n");

    for(i = 0; i < n; i++)
    {
        found = 0;

        for(j = 0; j < f; j++)
        {

```



```

    if(frames[j] == pages[i])
    {
        found = 1;
        break;
    }
}

if(found == 0)
{
    /* Check empty frame */
    pos = -1;

    for(j = 0; j < f; j++)
    {
        if(frames[j] == -1)
        {
            pos = j;
            break;
        }
    }

    /* If frames full */
    if(pos == -1)
    {
        farthest = -1;

        for(j = 0; j < f; j++)
        {
            index = -1;

            for(k = i + 1; k < n; k++)
            {
                if(frames[j] == pages[k])
                {
                    index = k;
                    break;
                }
            }

            if(index == -1)
            {
                pos = j;
                break;
            }

            if(index > farthest)
            {

```

```

        farthest = index;
        pos = j;
    }
}

frames[pos] = pages[i];
faults++;
}

printf("Page %d -> ", pages[i]);

for(j = 0; j < f; j++)
{
    if(frames[j] != -1)
        printf("%d ", frames[j]);
}

printf("\n");
}

printf("Total Page Faults = %d\n", faults);
}

/* ----- MAIN ----- */
int main()
{
    int n, f, i;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");

    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    FIFO(n, f);
    LRU(n, f);
    OPTIMAL(n, f);

    return 0;
}

```

Output:

```

C:\Users\Abhis\OneDrive\De
ABHISHEK SAVALI ---1BM24CS013
Enter number of pages: 21
Enter reference string:
5
0
1
0
2
3
0
2
4
3
3
2
0
2
1
2
7
0
1
1
0
Enter number of frames: 3

===== FIFO =====
Ref    F1    F2    F3    Status
5      5      -      -      Fault
0      5      0      -      Fault
1      5      0      1      Fault
0      5      0      1      Hit
2      2      3      1      Fault
3      2      3      1      Fault
0      2      3      0      Fault
2      2      3      0      Hit
4      4      3      0      Fault
3      4      3      0      Hit
3      4      3      0      Hit
2      4      2      0      Fault
0      4      2      0      Hit
2      4      2      0      Hit
1      4      2      1      Fault
2      4      2      1      Hit
7      7      2      1      Fault
0      7      0      1      Fault
1      7      0      1      Hit
1      7      0      1      Hit
0      7      0      1      Hit
FIFO Page Faults = 11

===== LRU =====
Ref    F1    F2    F3    Status
5      5      -      -      Fault
0      5      0      -      Fault
1      5      0      1      Fault
0      5      0      1      Hit
2      2      0      1      Fault
3      2      0      3      Fault
0      2      0      3      Hit
2      2      0      3      Hit
4      2      0      4      Fault
3      2      3      4      Fault
3      2      3      4      Hit
2      2      3      4      Hit
0      2      3      0      Fault
2      2      3      0      Hit
1      2      1      0      Fault
2      2      1      0      Hit
7      2      1      7      Fault
0      2      0      7      Fault
1      1      0      7      Fault
1      1      0      7      Hit
0      1      0      7      Hit
LRU Page Faults = 12

```